

Shape Truthful Clustering (STC) — Converged Design for LoManLe Integration

Status (2026-08-05). This document records the design agreed in discussion between Asis and Claude on this date. Nothing described here is implemented yet. It supersedes the earlier, simpler version of this file (single density/tangent observable tuple, no explicit API contract). For the general, density-only STC prototype’s own algorithm (unaffected by anything below except the implementation-quality notes in §5), see `misc/STC_current_algorithm_draft.md`. For LoManLe’s own current (pre-STC) implementation, see `misc/LoManLe.md` and `src/lomanle.F90` — that document will be updated to reflect STC once this is actually built and wired in, not before.

1. Generic STC algorithm (implementation-agnostic API)

STC identifies sets of vectors (“ensembles”) in a multivariate vector space via a procedure inspired by statistical-physics renormalization: grow a region from a seed while monitoring a macroscopic “observable,” and stop once further growth would change that observable beyond a threshold — the accepted ensemble is a “fixed point” of the growth operator.

This is deliberately **not** implemented as a single Fortran generic interface: different concrete instantiations (density-based, LoManLe’s tangent/dimension/gap-based, and future ones) are expected to differ in argument types and counts, not just behavior. The contract below is conceptual — every instantiation follows this five-step shape, with each step’s concrete mechanism pluggable:

1. **Identify seed vectors.** Each seed starts its own ensemble of size one.
2. **Grow an existing ensemble.** Default mechanism: expand by a fixed surface radius using the k-d-tree. Other growth mechanisms are explicitly anticipated (e.g. using metadata similarity to add vectors that are ambient-distant but metadata-close).
3. **Compute the observable** for the grown ensemble. Concrete content is pluggable (density statistics for the general prototype; (U_t, d_t, G_t) and derived quantities for the LoManLe flavor, §2 below). Keep either the last S observables or the full trajectory, so that change-over-iterations can be computed at any point.
4. **Accept or reject the growth**, by analyzing at least the current and previous observable. If rejected, the *penultimate* ensemble is the final result for that seed, and the algorithm moves on to the next seed (seeds may run in parallel, since each grows independently until its own acceptance decision).
5. **Reconcile ensembles:** identify intersections between the accepted ensembles. Whether/how to merge on top of that is pluggable per instantiation —

the LoManLe flavor (§2) does **not** merge; it only reports intersections, since LoManLe’s own downstream machinery (Step 7/8’s membership graph, Step 10’s stitching) already does reconciliation.

2. The LoManLe-flavored instantiation

2.1 Seed selection

Sequential, density-ranked, **excluding points already covered** by an already-accepted ensemble — i.e. the same greedy discipline LoManLe’s current `select_atlas_anchors` uses (Step 4/5 today), not independent/parallel seeding across the whole density ranking. This is what prevents many near-duplicate ensembles from being seeded in the same dense region; reconciliation (2.5) does not deduplicate, so this has to be handled at seed time.

2.2 Growth

Fixed-radius surface expansion via the k-d-tree (`src/k_d_tree.F90`’s `kd_tree` module — see §5). The radius is **not** a single dataset-wide value. It is derived per seed, using exactly the computation LoManLe’s `grow_one_point_neighborhood` already performs as `local_scale_i`:

$$\text{local_scale} = \begin{cases} \text{distance to the } \frac{k_{\min}+1}{2}\text{-th nearest neighbor,} & k_{\min} \text{ odd} \\ \frac{1}{2} (\text{distance to the } \frac{k_{\min}}{2}\text{-th} + \text{distance to the } (\frac{k_{\min}}{2} + 1)\text{-th nearest neighbor}), & k_{\min} \text{ even} \end{cases}$$

i.e. the **median distance among the seed’s own $k_{\min}$ nearest neighbors** — robust (median, not mean), computed once per seed from that seed’s local neighborhood, not once globally. This directly replaces `calculate_density_radius`’s single global 15th-percentile radius, which would otherwise reproduce the exact “one fixed scale is wrong across a manifold with varying density/curvature” problem LoManLe’s Phase II/III redesign already exists to solve. This computation now lives in one place only (STC’s growth step); there is no separate “LoManLe local scale” vs. “STC local scale.”

2.3 Observable

`$0_t = (U_t, d_t, G_t)$` plus everything else obtainable for free from the same eigendecomposition, so that LoManLe’s Step 6 (`compute_anchor_svd`) becomes fully redundant rather than partially:

- `$U_t$`: tangent basis (top-`$d_t$` eigenvectors).
- `$d_t$`: inferred intrinsic dimension.
- `$G_t$`: spectral-gap statistic, $\lambda_{d_t}/(\lambda_{d_t+1}+\epsilon)$.
- center (the ensemble’s mean vector — already computed to form the covariance).

- `normal_error` (mean squared residual off the tangent subspace — the “normal” eigenvalues, already computed, not currently reused).
- `tangent_scales` (per-tangent-direction extent — already computed alongside `$U_t$` in the current code).
- `$\Sigma_{\perp}$` (the anisotropic normal-space covariance, §4 below) — **also free**: `dsyev` already returns the *full* eigendecomposition (all `$D$` eigenvalue/eigenvector pairs), of which only the top `$d_t$` are currently kept as the tangent basis. The remaining `$D-d_t$` pairs, already computed and already discarded today, are exactly `$\Sigma_{\perp}$`’s eigendecomposition in the ambient basis. Nothing extra needs computing to get it.

2.4 Accept / reject (growth stability — within one ensemble’s own iterations)

$$\delta_U < \tau_U, \quad \delta_d = 0, \quad \delta_G < \tau_G,$$

comparing the current growth iteration’s observable against the previous one for the *same* ensemble — this is the direct generalization of LoManLe’s existing tangent-stability check (Phase III), now also requiring the inferred dimension itself to be stable, which is what resolves LoManLe’s previously-open “local intrinsic dimension inference” item. **This is unrelated to, and unaffected by, the cross-anchor stitch-gate correction in §4** — it governs when a single ensemble stops growing, not whether two different ensembles should be considered the same manifold.

2.5 Reconcile

Report ensemble-pair intersections only. No merging. Feeds directly into what is currently Step 7/8 (`build_membership_matrix`, the intersection graph) unchanged.

3. What this replaces in LoManLe’s pipeline

- **Steps 1-5b (Phases II-V today) → fully replaced** by §2’s seed/grow/observable/accept/reconcile loop.
- **Step 6 (`compute_anchor_svd`) → redundant, can be deleted.** Every quantity it computes is already produced as a byproduct of §2.3’s observable.
- **Steps 6.5-9 (membership matrix, intersection graph, BFS labels) → unchanged**, consuming STC’s ensembles and reported intersections the same way they consume today’s `is_anchor_mask/sphere_radii`.
- **Step 10 (stitching) and Phase XII (backbone/MST) → gain a new pair-level gate**, described next. This is new logic, not a replacement of existing logic.

4. The pair-level stitch/connectivity gate

Resolves LoManLe.md §13 ($\Sigma_{\perp}$, previously unimplemented) and §14 (the noise-tube stitch decision, previously unimplemented — the “Potential Stitching Experiment”). Evaluated **once per candidate anchor pair** (a, b) , not once per point — reusing the same pair-enumeration technique Step 8 and the MST’s Tier-1/Tier-2 candidate discovery already use.

Gate:

1. **Ensemble/sphere overlap** — cheap necessary prefilter, already available (Step 8’s candidate graph / the MST’s Tier-2 sphere-overlap fallback). This also satisfies the design’s original “non-gap requirement” (real observations must populate the junction) for free: shared member points between two ensembles *are* real observations at the junction, so no separate check is needed.
2. **Noise-tube compatibility:** $D_{ab}^2 = \Delta_{ab}^{\text{top}} \Sigma_{\perp, ab} + \Delta_{ab} \leq \tau_{\text{noise}}$, with $\Sigma_{\perp, ab} = \Sigma_{\perp, a} + \Sigma_{\perp, b}$ (both expressed in a compatible ambient-space representation) and Σ the Moore-Penrose pseudoinverse. **This is the actual required gate.**

Explicitly not gated by tangent angle. An earlier version of this design proposed also requiring tangent-angle compatibility ($\cos \theta_j = s_j(U_a^{\text{top}} U_b)$, small angle) as a joint condition. This was wrong: at a genuine bifurcation, the angle between the two outgoing branches can be large, even orthogonal, and a required-compatibility framing would incorrectly reject real junctions. The angle is still worth computing and recording (U_t is free from §2.3) — small angle suggests **continuation**, large/divergent angle at a passing gate suggests **furcation** — but purely as a classification *label* for downstream use, never as a reason to reject a pair that already passed 1 and 2.

Where it plugs in:

- **Step 10 (stitch_multi_anchor_point):** a point’s covering anchors should only be blended together if their anchor pair passes the gate. Today, blending happens unconditionally for any `anchor_count(i) >= 2`. If a point’s covering anchors fail the gate, it should not be forced onto a single blended position between manifolds the geometry doesn’t actually support treating as one.
- **Phase XII (MST candidate-edge generation):** candidate pairs failing the gate should be rejected or downgraded before Tier-1/Tier-2 offer them to Kruskal. Today, furcation is inferred purely *after the fact* from MST anchor degree (≥ 3 = junction) — a topological proxy with no geometric test behind it. Gating candidate edges directly makes this one consistent decision instead of two independent, occasionally-disagreeing mechanisms (Step 10 blending unconditionally vs. Phase XII inferring structure from degree alone).

5. Implementation notes carried over regardless of variant

These apply to both the general density-only prototype and the LoManLe flavor, and were identified from the code already on the `shape-truthful-clustering` branch:

- **Use only `src/k_d_tree.F90`'s `kd_tree` module** (`build_kd_index`, `kd_knn_query`, `kd_range_query_mask`, `kd_range_query_list`) — already performance-hardened for LoManLe's own use. Do not extend or depend on `f42_kd_tree.F90`'s parallel/divergent `vicinity_vectors`; that module should not be carried forward.
- **Avoid dense $N \times N$ masks.** `calculate_labels_as_density_alloc`'s `tmp_vicinity_mask(n_vectors, n_vectors)` is $O(N^2)$ memory — tens of GB at LoManLe's own benchmark scale ($N=50\{, \}000$). Use per-seed/per-thread scratch (as LoManLe's own `!$omp parallel do` regions already do) or sparse/CSR structures (as Step 7/8 already do) instead.
- **Use `!$omp parallel do`, not do concurrent, for any loop calling an external procedure** (k-d-tree queries, `dsyev`). Confirmed directly for LoManLe (`misc/smoothing_experiments.md` §18.4): gfortran's do concurrent auto-parallelizer refuses to parallelize a loop whose body calls an external procedure, even a pure one, regardless of `-ftree-parallelize-loops` — the resulting binary links no `libgomp` symbols at all. `calculate_labels_as_density_helper`'s do concurrent loop calling `vicinity_vectors` will silently run single-threaded as written.

6. Open items — not yet decided

- Exact thresholds (τ_{noise} , τ_U , τ_G , the density prototype's α_{MAD} / α_{accept}) all need empirical tuning; no default values are agreed yet for the LoManLe flavor.
- Whether Step 6 is deleted outright once STC supplies everything it computed, or kept as a thin consistency check during a transition period.
- The exact code-level wiring of the §4 gate into Step 10 and Phase XII (data flow only sketched above, not yet designed at the subroutine-signature level).
- STC's own module/file layout and naming — whether it replaces `tox_shatter_cluster_data.F90` in place or lands as a fresh module, and how the generic five-step contract (§1) is expressed across the two concrete instantiations without Fortran generic interfaces (per Asis's explicit constraint).